AI FastShop Theme Generation — All Prompts

> Extracted from the Python FastAPI backend at `D:\Universell Projects\universell-ai-apis`. Scope: every LLM prompt used by the FastShop theme generator (`/api/generate-website` + `/api/edit-theme-v2` + `/api/edit-theme` legacy). Landing page generator prompts are documented separately in `AI_LANDING_PAGE_PROMPTS.md`.

The theme generator runs a **5-node LangGraph workflow** (wired in `app/workflow_graph.py`). It does **NOT generate HTML** — the CI4 FastShop frontend handles all rendering. The workflow emits JSON only: design tokens + content + image prompts.

```
START → business_gathering → [if ready] → design_tokens → page_structure
                                        → content_generation → image_prompts → END
```

End-to-end timing target: ~100–135 seconds.

Phase	LLM call?	Where the prompt lives
business_gathering	Yes — 5–6 conversational turns	`app/workflow_node/business_gathering/prompts.p` (SystemMessage)
design_tokens	Yes — 1 call	`app/workflow_node/design_token_node/node.py` (inline f-string)
page_structure	Yes — 1 call	`app/workflow_node/page_structure_node/node.py` (inline f-string)
content_generation	Yes — 1 call (the big one)	`app/workflow_node/content_generation_node/node` (inline f-string)
image_prompts	**No** — Python template only	`app/workflow_node/image_prompt_node/node.py` (`_build_image_prompts_template`)
`/api/edit-theme-v2`	Yes — tool-calling agent	`app/edit_agent/agent.py` (`SYSTEM_PROMPT_TEMPLATE`) + `app/edit_agent/tools.py` (`TOOL_SPECS`)
`/api/edit-theme` (legacy)	Yes — full JSON rewrite	`app/main.py` inline `ThemeEditSignature`
`/api/apply-tool`	**No** — direct dispatch	n/a — no prompt

LLM: Azure OpenAI `gpt-5.1-codex-mini` (reasoning model — uses `responses.create`, NOT `chat.completions.create`). `reasoning.effort="low"` is set on most calls to prevent reasoning tokens from starving the visible output.

> **NOTE on `planning_node` and `html_generation`** — both modules still exist on disk under `app/workflow_node/` but are explicitly **commented out** of `workflow_graph.py` (the old workflow that

generated PHP/HTML directly). Their prompts are NOT used by the current theme generator and are not documented here. The CI4 FastShop frontend now renders from the JSON theme output instead.

---

1. BUSINESS GATHERING (Node 1 of 5)

The conversational onboarding chat. Asks ONE question per turn (color → secondary → tagline → hero → CTA → done). Uses Azure `responses.create` directly (not DSPy) because the codex-mini reasoning model requires that API.

File: `app/workflow_node/business_gathering/prompts.py` **Constant:** `BUSINESS_GATHERING_SYSTEM_PROMPT` (LangChain `SystemMessage`) **LLM call:** `business_gathering_client.responses.create(...)` in `business_gathering/node.py` **Config:** `max_output_tokens=8000`, `reasoning={"effort": "low"}`, `text={"format": {"type": "json_object"}}`

System prompt (verbatim)

```
You are a senior e-commerce website planner AI.
Your goal is to gather branding and shop configuration information through a STEP-BY-STEP co

IMPORTANT CONTEXT:
- The website will be integrated into a CI4 (CodeIgniter 4) CRM system
- Product data, categories, and subcategories already exist in the CRM database
- You do NOT need to ask about products, categories, merchant ID, FAQ, or contact info -
these are handled automatically
- The frontend handles color picking via a visual color picker widget

YOUR STEP-BY-STEP FLOW - Ask ONE question per response, in this EXACT order:

IMPORTANT: EVERY question MUST include an "Example:" in the question text to guide the user.

STEP 1 (after user's first message describing their business):
- Extract: brand_name, business_type from what they said
- Ask ONLY: "What is your brand's primary color? Example: <business-specific color suggestio
- Set ready: false

STEP 2 (after user provides primary color):
- Store the primary color in business_plan
- Ask ONLY: "What secondary/accent color would you like? Example: <business-specific suggest
- Set ready: false

STEP 3 (after user provides secondary color):
- Store the secondary color in business_plan
```

```
- Ask ONLY: "What tagline would you like for your store? Example: <business-specific tagline
- Set ready: false

STEP 4 (after user provides tagline):
- Store the tagline in business_plan
- Ask ONLY: "What should the main headline and subheadline on your hero banner say? Example:
'<business-specific headline, e.g., Warmly Baked Treats Daily>', Subheadline -
'<business-specific subheadline, e.g., From crusty loaves to delicate pastries, made fresh e
- Set ready: false

STEP 5 (after user provides hero headline/subheadline):
- Store hero_headline and hero_subheadline in business_plan
- Ask ONLY: "What text should the main call-to-action button say? Example: <business-specifi
- Set ready: false

STEP 6 (after user provides CTA button text):
- Store hero_cta_text in business_plan
- You now have ALL required information
- Set ready: true with the COMPLETE finalized business_plan

HOW TO DETERMINE WHICH STEP YOU ARE ON:
Count the number of user messages in the conversation (excluding the system):
- 1 user message (initial description) → you are responding to STEP 1 → ask for primary colo
- 2 user messages (description + primary color) → STEP 2 → ask for secondary color
- 3 user messages → STEP 3 → ask for tagline
- 4 user messages → STEP 4 → ask for hero headline/subheadline
- 5 user messages → STEP 5 → ask for CTA button text
- 6 user messages → STEP 6 → set ready: true

CRITICAL RULES:
- Ask EXACTLY ONE question per response
- NEVER ask for FAQ, contact info, merchant ID, or product details
- NEVER ask "Please provide more details" - always ask the SPECIFIC next question from the s
- NEVER set ready=true until STEP 6 (after CTA text is provided)
- If user says "skip" for any step, use a smart default and move to next step
- If user says "ready to generate" or "go ahead" at any step, fill ALL remaining steps with
- Accumulate ALL gathered information in business_plan at every step

BUSINESS PLAN FORMAT (accumulated progressively, key=value pairs separated by periods):
brand_name=Sweet Delights. business_type=bakery. primary_color=#f97316. secondary_color=#fef

CRITICAL OUTPUT RULES:
- Respond with ONLY valid JSON
- Output must start with { and end with }
- NO markdown code fences
- NO extra text before or after the JSON
```

```
JSON FORMAT (ready=false):
{
  "ready": false,
  "questions": ["The single question for this step"],
  "business_plan": "Accumulated key=value pairs so far"
}

JSON FORMAT (ready=true, STEP 6 only):
{
  "ready": true,
  "questions": [],
  "business_plan": "Finalized: brand_name=... business_type=... primary_color=... secondary_
}
```

Server-side safety net (Python, not prompt)

If `user_msg_count >= 6` and the LLM still hasn't set `ready=true`, `business_gathering/node.py` forces it to true with the accumulated business plan. This guards against the model getting stuck in a loop.

Auto-build fallback (used when user types "ready to generate" early)

`_build_auto_business_plan()` in `business_gathering/node.py` synthesizes a plausible business plan from keyword inference (fruit/grocery → green palette, blue/tech → blue, dark/luxury → dark). Not a prompt — it's a deterministic generator.

---

2. DESIGN TOKEN GENERATION (Node 2 of 5)

Generates the `theme_code` JSON object that maps **directly to the CI4 `user.theme_code` column** — colors, fonts, sizes. The schema must match `saveStoreAppearance` in `Admin/Fshop.php`.

File: `app/workflow_node/design_token_node/node.py` **DSPy signature class:** `DesignTokenSignature` **DSPy module:** `DesignTokenGenerator` (uses `dspy.Predict`)

DSPy signature docstring

```
Generate design tokens (theme_code) for an eCommerce store.
```

Inline prompt (built in `DesignTokenGenerator.forward`)

```
Generate a theme_code JSON object for an eCommerce website based on this business descriptio
```

```
{business_description}

You MUST return ONLY a valid JSON object with EXACTLY these keys (no extras, no missing):

{
  "theme_background_color": "#HEXCODE",
  "button_background_color": "#HEXCODE",
  "button_font_color": "#HEXCODE",
  "link_highlight_color": "#HEXCODE",
  "thumbnail_background_color": "#HEXCODE",
  "enable_product_border": "0" or "1",
  "product_border_color": "#HEXCODE",
  "product_card_bg_color": "#HEXCODE",
  "product_card_text_color": "#HEXCODE",
  "primary_font_family": "one of: {VALID_FONT_FAMILIES - see below}",
  "primary_font_size": "one of: {VALID_FONT_SIZES - see below}",
  "primary_font_color": "#HEXCODE"
}

RULES:
- ALL color values must be uppercase 7-char hex: #RRGGBB
- primary_font_family must be one of the listed values
- primary_font_size must be one of the listed values (em-based)
- enable_product_border must be "0" or "1"
- Choose colors that match the brand/business type
- Ensure good contrast (dark text on light bg, light text on dark buttons)
- Return ONLY the JSON object, no markdown, no explanation
```

Allow-lists injected into the prompt

```
VALID_FONT_FAMILIES = [
    "arial", "helvetica", "times_new_roman", "georgia", "verdana",
    "courier_new", "trebuchet_ms", "comic_sans_ms", "impact", "palatino",
    "garamond", "bookman", "tahoma", "lucida_sans", "roboto", "open_sans",
    "lato", "montserrat", "poppins", "raleway",
]

VALID_FONT_SIZES = [
    "0.5", "0.625", "0.75", "0.875", "1", "1.125", "1.25",
    "1.375", "1.5", "1.625", "1.75", "2", "2.25", "2.5", "2.75",
]
```

Post-LLM Python overrides

After the LLM responds, `_extract_colors_from_business_plan()` parses `primary_color=#...`, `secondary_color=#...`, `accent_color=#...` from the

business plan text. **The user's chosen colors always win** over whatever the LLM produced — `_build_theme_from_colors()` rebuilds the theme around them and even auto-flips button text to white/dark based on primary luminance.

---

3. PAGE STRUCTURE (Node 3 of 5)

Picks **which sections appear on each page and in what order**. Output maps to CI4 `shop_settings` (type 1–5).

File: `app/workflow_node/page_structure_node/node.py` **DSPy signature class:** `PageStructureSignature` **DSPy module:** `PageStructureGenerator` (uses `dspy.Predict`)

DSPy signature docstring

```
Generate page section structure for an eCommerce website.
```

Inline prompt (built in `PageStructureGenerator.forward`)

```
Based on this business description, decide which sections each page should have and in what

{business_description}

Available sections per page:
{sections_desc}     # serialized AVAILABLE_SECTIONS dict - see below

Return ONLY a JSON object like this:
{
  "home": ["hero_banner", "category_nav", "featured_products", "shop_by_category", "product_
  "product_detail": ["breadcrumb", "product_gallery", "product_info", "product_tabs", "relat
  "faq": ["faq_hero", "faq_accordion", "contact_cta"],
  "checkout": ["customer_info", "shipping_method", "payment_method", "order_summary"],
  "login": ["login_form", "social_login", "auth_links"],
  "register": ["register_form", "auth_links"]
}

RULES:
- Use ONLY section names from the available sections list
- home page MUST include "category_nav" and "product_grid"
- checkout/login/register sections are fixed (include all of them)
- Order sections by importance for the business type
- Return ONLY JSON, no markdown, no explanation
```

`AVAILABLE_SECTIONS` (injected into prompt as `sections_desc`)

```
{
  "home": {
    "hero_banner": "Hero banner with headline, subheadline, CTA button",
    "category_nav": "Category navigation bar (always included, uses $categories)",
    "featured_products": "Featured products grid (shop_settings type=3)",
    "shop_by_category": "Shop by category cards (shop_settings type=1)",
    "browse_subcategories": "Browse subcategories section (shop_settings type=2)",
    "product_grid": "All products grid with filters (uses $results)",
    "testimonials": "Customer testimonials/reviews",
    "cta_banner": "Call-to-action banner",
    "scrolling_text": "Scrolling banner text (uses user.banner_text)",
  },
  "product_detail": {
    "breadcrumb": "Product breadcrumb navigation",
    "product_gallery": "Product image gallery with thumbnails",
    "product_info": "Product title, price, stock, add-to-cart",
    "product_tabs": "Tabbed info (description, reviews, details)",
    "related_products": "Related or recommended products",
  },
  "faq": {
    "faq_hero": "FAQ page header section",
    "faq_accordion": "FAQ questions in accordion format",
    "contact_cta": "Contact us call-to-action",
  },
  "checkout": {
    "customer_info": "Customer information form",
    "shipping_method": "Shipping method selection",
    "payment_method": "Payment method selection",
    "order_summary": "Order summary with totals",
  },
  "login": {
    "login_form": "Login form with email/password",
    "social_login": "Google OAuth button",
    "auth_links": "Register and forgot password links",
  },
  "register": {
    "register_form": "Registration form with details",
    "auth_links": "Login link for existing users",
  },
}
```

Required sections (enforced server-side, not in prompt)

```
FIXED_SECTIONS = {
    "home": ["category_nav"],
    "checkout": ["customer_info", "shipping_method", "payment_method", "order_summary"],
    "login": ["login_form", "auth_links"],
    "register": ["register_form", "auth_links"],
}
```

Plus: `home` always has `product_grid` appended if the LLM omitted it.

---

4. CONTENT GENERATION (Node 4 of 5) — the biggest call

Generates all text content for the page (hero copy, FAQ, CTA, reviews, trust stats, etc.), the `design_style`, and the initial `section_order`. **This is the most expensive LLM call (~85s).**

File: `app/workflow_node/content_generation_node/node.py` **DSPy signature class:** `ContentSignature` **DSPy module:** `ContentGenerator` (uses `dspy.Predict`)

DSPy signature docstring

```
Generate website text content and layout for an eCommerce store.
```

Inline prompt (built in `ContentGenerator.forward`)

Before sending, the business description is cleaned of `[merchant_id=...]` tags and `Custom (#hex)` color-picker noise.

```
Generate website text content for an online store.

Business description: {clean_desc}

Return ONLY valid JSON with this structure:
{
  "brand_name": "The business name",
  "tagline": "5-10 word tagline",
  "hero_title": "4-8 word engaging headline",
  "hero_subtitle": "1-2 sentence subtitle",
  "hero_cta_text": "2-3 word button text",
  "header_description": "One-line description",
  "footer_description": "Short tagline",
  "banner_text": "",
  "brand_voice": "warm",
  "faq": [
```

```
    {"question": "Industry-specific question", "answer": "Helpful answer"},
    {"question": "Another question", "answer": "Answer"},
    {"question": "Third question", "answer": "Answer"},
    {"question": "Fourth question", "answer": "Answer"},
    {"question": "Fifth question", "answer": "Answer"}
  ],
  "cta_title": "CTA headline",
  "cta_subtitle": "CTA description",
  "cta_button_text": "CTA button text",
  "about_title": "About heading",
  "about_description": "100-150 word story about the business, writing in first person (we)
  "contact_email": "support@brand.com",
  "contact_phone": "+1 (555) 000-0000",
  "contact_address": "City, State",
  "design_style": "warm-cozy",
  "section_order": ["hero_banner", "features_usp", "shop_by_category", "featured_products",
  "features_usp_heading": "Section title",
  "features_usp_subheading": "Subheadline",
  "features_usp_items": [
    {"title": "Benefit 1", "description": "8-15 word description", "icon": "star"},
    {"title": "Benefit 2", "description": "Description", "icon": "truck"},
    {"title": "Benefit 3", "description": "Description", "icon": "heart"}
  ],
  "trust_stats": [
    {"value": "Business-specific number", "label": "Label"},
    {"value": "Another metric", "label": "Label"},
    {"value": "Third stat", "label": "Label"}
  ],
  "trust_stats_heading": "Stats section heading",
  "trust_stats_subheading": "Subheadline",
  "reviews_heading": "Reviews section title",
  "reviews_items": [
    {"title": "Short review title", "text": "25-40 word testimonial", "name": "Full Name"},
    {"title": "Title", "text": "Testimonial", "name": "Full Name"},
    {"title": "Title", "text": "Testimonial", "name": "Full Name"}
  ],
  "shipping_heading": "Shipping heading",
  "shipping_subheading": "Subheadline",
  "shipping_items": [
    {"title": "Benefit", "description": "Detail"},
    {"title": "Benefit", "description": "Detail"},
    {"title": "Benefit", "description": "Detail"}
  ]
}

Guidelines:
```

```
- brand_voice values: warm, modern, bold, playful, premium
- design_style values: warm-cozy, modern-minimal, bold-vibrant, elegant-luxury
- Match brand_voice and design_style to business type (bakery=warm/warm-cozy, tech=modern/mo
- Make trust_stats specific to the business (e.g., years in business, products made, reviews
- Use diverse realistic full names for reviews (e.g., "Maria Gonzalez", "Alex Kim", "James (
- Write about_description in authentic voice with specific details (year, location, craft)
- Generate exactly 5 FAQ items relevant to the business type
- Icons for features_usp_items: star, truck, shield, heart, clock, bread, leaf, food
- Return valid JSON only, no markdown, no commentary
```

Heavy post-LLM enrichment (no prompt — pure Python)

After the LLM responds, content_generation_node does a LOT of deterministic work:

1. **User content override** — `_extract_content_from_business_plan()` re-injects the exact brand_name / tagline / hero text / FAQ that the user provided in business_gathering. The LLM's versions are discarded if the user supplied their own.
2. **Industry archetype pick** — `archetypes.pick_archetype(business_description, fallback_style=...)` keyword-matches against 16 presets. Each archetype contributes:
   - `design_style` (overrides LLM)
   - `typography` pair (overrides default)
   - `kit` axes (spacing_density, corner_radius, shadow_depth, motion, decoration)
   - `variants` per section
   - `section_emphasis` (re-orders sections)
   - `image_mood` (passed to image_prompts node)

   Archetype keyword buckets (in match order):

```
artisan_bakery     ["bakery", "bread", "pastry", "patisserie", "cake", "dough"]
farm_fresh         ["farm", "fresh produce", "organic", "vegetable", "fruit", "grocery"
vintage_cafe       ["cafe", "coffee", "espresso", "latte", "tea house", "teashop", "cha
restaurant_elegant ["restaurant", "fine dining", "bistro", "eatery", "cuisine"]
modern_boutique    ["boutique", "apparel", "clothing store", "fashion"]
luxury_fashion     ["luxury", "designer", "couture", "atelier", "high-end", "premium fa
streetwear_edge    ["streetwear", "urban", "skate", "hype", "sneaker"]
tech_minimal       ["saas", "software", "tech startup", "app", "platform", "b2b", "deve
gadget_bold        ["electronics", "gadget", "device", "smartphone", "tablet", "tech st
wellness_calm      ["wellness", "yoga", "meditation", "spa", "mindfulness", "holistic"]
fitness_energetic  ["fitness", "gym", "crossfit", "athlet", "workout", "training", "spo
medical_trust      ["medical", "clinic", "pharmacy", "healthcare", "doctor", "dental",
beauty_chic        ["beauty", "cosmetic", "makeup", "skincare", "salon", "barber", "hai
home_artisan       ["furniture", "home decor", "interior", "homeware", "handcrafted", .
```

```
creative_playful   ["kids", "toy", "art supply", "craft", "stationery", "creative", "ho
service_professional ["consulting", "service", "agency", "studio", "legal", "accounting
```

3. **Hash-seeded variant + kit pick** — `_build_layout_variants()` and `_build_design_kit()` pick from style-specific preference lists, seeded by an MD5 of the business description. Same business regenerates the same layout; different businesses get variety. Archetype-curated choices win over hash picks.
4. **CRM-aware tuning** — `_apply_merchant_facts()` reads `merchant_facts` injected from CI4 (read-only) and rewrites variants/section_order:
   - `product_count` 30 → `products` variant = `scroll_strip`
   - `product_count` 5 → `products` variant = `wide_details`
   - `category_count` 3 or 8 → `categories` variant = `grid`
   - `avg_price` 200 or `max_price` 1000 → `spacing = airy`, `shadow = elevated`
   - No featured products → drop `featured_products` section
   - No hero image → hero variant = `centered`
   - `product_count` 3 (service business) → promote `about_us` + add `trust_stats`
   - **Invariant:** if `product_count > 0`, `about_us` is forced below `featured_products` and `shop_by_category`.
5. **`section_layout` is built deterministically in Python** (`_build_section_layout()`) from the validated content fields. The LLM never produces this array — it just provides flat content.

Valid enums (Python-side validation after LLM responds)

```
VALID_DESIGN_STYLES = ["modern-minimal", "warm-cozy", "bold-vibrant", "elegant-luxury"]
VALID_SECTION_TYPES = [
    "hero_banner", "features_usp", "shop_by_category", "featured_products",
    "browse_categories", "about_us", "trust_stats", "shipping_info", "reviews", "promo_banne
]
```

Variant options (per-section)

```
hero       = ["split", "centered", "card_overlay"]
categories = ["carousel", "grid", "masonry"]
products   = ["magazine", "wide_details", "scroll_strip"]
about      = ["editorial", "stacked", "offset_frame"]
reviews    = ["magazine_cards", "quote_wall", "carousel"]
footer     = ["classic", "minimal"]
```

Typography pairings by design_style

```
_TYPOGRAPHY_PAIRINGS = {
    "warm-cozy":      [("playfair_display","lora"), ("dm_serif_display","nunito"), ("cormora
```

```
    "modern-minimal": [("inter","inter"), ("manrope","manrope"), ("work_sans","inter"), ("dm
    "bold-vibrant":   [("oswald","nunito"), ("montserrat","montserrat"), ("raleway","open_sa
    "elegant-luxury": [("cormorant_garamond","lora"), ("playfair_display","raleway"), ("dm_s
}
```

---

5. IMAGE PROMPTS (Node 5 of 5) — NO LLM CALL

The 3 image prompts (brand_logo, hero_banner, about_us) are built from a **Python template** seeded by the business context, archetype `image_mood`, and `brand_voice`. **There is no LLM call here** — this was previously a reasoning-model call that took 150–340s; replaced with instant template interpolation.

The 4-image LLM signature (`ImagePromptSignature`) is still defined in the file but **dead code in the current workflow**.

File: `app/workflow_node/image_prompt_node/node.py` **Function:** `_build_image_prompts_template(content, business_description)`

Mood/lighting keyed by `brand_voice`

```
mood_map = {
    "warm":         ("golden hour",              "warm natural light",  "cozy editorial"),
    "luxury":       ("soft studio light",        "refined editorial",   "cinematic"),
    "bold":         ("dramatic directional light","high contrast",      "bold editorial"),
    "modern":       ("soft diffused daylight",   "clean minimal",       "cinematic"),
    "playful":      ("bright natural light",     "vibrant lifestyle",   "dynamic"),
    "minimal":      ("soft diffused light",      "clean negative space","editorial"),
    "professional": ("soft studio light",        "refined editorial",   "cinematic"),
}
```

Shared `NO_TEXT_SUFFIX` appended to hero + about prompts

```
Absolutely no text, no words, no letters, no signage, no writing, no typography, no labels,
```

Generated prompts (templates)

brand_logo (1024×1024):

```
Premium minimalist brand logo for '{brand}' - {biz_ctx}.
Elegant wordmark with a subtle icon motif relevant to the business,
centered on pure white background, {style} typography, scalable and professional.
No mockups, no packaging, no background clutter.
```

hero_banner (1536×1024 — landscape, matches the frontend split hero wide card):

```
Wide editorial photograph of the actual products / signature items of a {biz_ctx} brand as t
```

```
styled tastefully in their natural environment (no portraits, no models, faces never the sub
{lighting}, cinematic rule-of-thirds composition, products grouped on the right two thirds l
{mood} tone, shallow depth of field, premium magazine quality.
[Visual mood: {archetype_mood}.]   ← only if archetype provided one
Absolutely no text, no words, no letters, no signage, no writing, no typography, no labels,
```

about_us (1024×1024 — square, matches the about two-column layout):

```
Authentic behind-the-scenes documentary photograph of a {biz_ctx} workspace or craftsmanshi


focus on the craft, tools, materials, or finished items (hands may appear but no faces, no p
{lighting}, candid composition, warm emotional connection, {mood} feel, genuine not staged.
[Visual mood: {archetype_mood}.]   ← only if archetype provided one
Absolutely no text, no words, no letters, no signage, no writing, no typography, no labels,
```

Image generation

Once prompts are built, all 3 images are generated in parallel via `asyncio.gather` against Azure `gpt-image-1.5` (90s per-image timeout). Filenames use short prefixes (`lg_`, `hb_`, `ab_`) so the compact `theme_code` JSON fits the CHAR(255) DB column.

Dead-code: the old `ImagePromptSignature` (kept in file but unused)

```
Generate image generation prompts for eCommerce website sections.
```

Old inline prompt is still in `ImagePromptGenerator.forward()` for reference — it asked for 4 sections (brand_logo, hero_banner, about_us, feature_image) via a reasoning LLM call. The current `_build_image_prompts_template` replaced it.

---

6. EDIT — `/api/edit-theme-v2` (primary edit endpoint, tool-calling agent)

The frontend's chat-based edit dispatches here. Uses Azure `responses.create` directly. The LLM cannot rewrite the theme JSON — it can only emit a JSON array of typed tool calls that Python dispatches.

File: `app/edit_agent/agent.py` **Function:** `run_edit_agent(theme, user_request)` **Config:** `max_output_tokens=6000`, `reasoning={"effort": "low"}`, `text={"format": {"type": "json_object"}}`

System prompt template

The `{tool_catalog}` placeholder is filled in by `_tool_catalog_for_prompt()` which renders all 16 tools in `TOOL_SPECS` as one-liners with their enum params.

```
You are a precise theme-editing agent for an e-commerce site builder.

You CANNOT modify the theme directly. You can only emit a JSON array of TOOL CALLS that I wi

AVAILABLE TOOLS:
{tool_catalog}

RULES:
1. Emit the MINIMUM number of tool calls needed. "Make the header darker" → ONE call to set_
2. You may emit MULTIPLE calls in a single turn if the request needs several changes (e.g. '
3. If a request is AMBIGUOUS (e.g. "make it nicer"), emit a single clarify call with 2-4 opt
don't guess.
4. For layout words:
   - "horizontal" / "side by side" / "two column" → set_section_variant(section="hero", vari
   - "vertical" / "stacked" / "full-width banner" → set_section_variant(section="hero", vari
   - "overlay" / "floating card" → set_section_variant(section="hero", variant="card_overlay
5. For "primary" / "brand" color use field="primary". For "darker" pick a hex darker than cu
6. Hex colors MUST be full 6-char hex like "#1A1A1A" (not shorthand).

OUTPUT FORMAT - respond with ONLY this JSON, no markdown fences, no explanation:
{
  "calls": [
    {"name": "<tool_name>", "args": {...}},
    ...
  ]
}

If the request is satisfiable with zero changes, return {"calls": []}.
```

User message (per request)

```
Current theme state (for reference):
{compact_theme_summary - JSON with design_style, colors, fonts, layout_config, variants, sec

User request: {user_request}
```

Tool catalog (16 typed tools, source of truth: `app/edit_agent/tools.py::TOOL_SPECS`)

Tool	Description (passed to LLM)	Required params
`set_section_variant`	Change the visual LAYOUT of a section. SEMANTIC MAP: 'horizontal'/'side-by-side'/'two-column'→hero=split; 'vertical'/'stacked'/'full-width banner'→hero=centered; 'overlay'/'floating card'→hero=card_overlay.	`section` (hero/categories/products/about/reviews/footer), `variant` (enum)
`set_color`	Change a color. field='primary' is the button/brand color. field='header' is the top nav. field='background' is the page body.	`field` (primary/secondary/button_text/background/header/text `hex`
`set_typography`	Change fonts. Provide heading_font, body_font, or both.	optional `heading_font`, `body_font` (14-font whitelist)
`set_spacing_density`	How roomy the sections feel. compact=tight, normal=default, airy=lots of whitespace.	`density` (compact/normal/airy)
`set_corner_radius`	How rounded corners are. sharp=0, subtle=4px, rounded=12px, pill=full.	`level` (sharp/subtle/rounded/pill)
`set_design_style`	Overall theme mood preset. Only use if the user explicitly asks for a full style change.	`style` (modern-minimal/warm-cozy/bold-vibrant/elegant-luxury)
`toggle_category_menu`	Top horizontal nav vs vertical left sidebar for categories.	`style` (horizontal/vertical)
`set_header_layout`	standard=logo-left/nav-right; centered=logo-centered/nav-below; minimal=logo+icons only.	`layout` (standard/centered/minimal)

Tool	Description (passed to LLM)	Required params
`set_product_grid_columns`	Number of products per row on the featured grid.	`columns` (2/3/4)
`set_footer_columns`	Number of columns in the footer.	`columns` (2/3/4)
`update_text`	Change a text content field on the site (headlines, taglines, about text, etc.).	`field` (hero_title/hero_subtitle/hero_cta_text/brand_name `value`
`add_section`	Add a section to the page. Optional position (0-based). Omits gracefully if section already present.	`section_type` (10-section enum), optional `position`
`remove_section`	Remove a section from the page.	`section_type`
`reorder_sections`	Reorder sections. Pass an array of section_type strings in the desired order. Any omitted sections are appended at the end.	`order` (array of section_type)
`set_custom_css`	ESCAPE HATCH for edge-case tweaks that no other tool covers. Pass raw CSS string. Use sparingly.	`css`
`clarify`	If user request is ambiguous, call this INSTEAD of guessing. Provides a question and 2-4 options for the user to pick from. This ends the edit (no changes applied until user answers).	`question`, optional `options` (array)

Enum values (referenced by tools above)

```
COLOR_FIELDS keys: primary, secondary, button_text, background, header, text, card_bg, card_
VARIANT_ENUMS:
  hero       → split | centered | card_overlay
  categories → carousel | grid | masonry
```

```
  products   → magazine | wide_details | scroll_strip
  about      → editorial | stacked | offset_frame
  reviews    → magazine_cards | quote_wall | carousel
  footer     → classic | minimal
DESIGN_STYLES: modern-minimal | warm-cozy | bold-vibrant | elegant-luxury
FONT_FAMILIES (14): poppins, roboto, montserrat, inter, playfair_display, lora, merriweathe
                    raleway, nunito, dm_serif_display, cormorant_garamond, work_sans, manrop
TEXT_FIELDS: hero_title, hero_subtitle, hero_cta_text, brand_name, tagline, header_descripti
             footer_description, about_title, about_description, cta_title, cta_subtitle, ct
SECTION_TYPES: hero_banner, features_usp, shop_by_category, featured_products, browse_catego
               about_us, trust_stats, shipping_info, reviews, promo_banner
```

---

7. EDIT — `/api/edit-theme` (LEGACY — full JSON rewrite)

The original edit endpoint. Kept for backwards compatibility but **not used by the current frontend** (which calls `/api/edit-theme-v2`). One large LLM call rewrites the entire theme JSON.

File: `app/main.py` (around line 1707) **Inline class:** `ThemeEditSignature` (DSPy) **Module:** `dspy.ChainOfThought(ThemeEditSignature)`

Signature docstring (the entire instruction set)

```
You are a PRECISE AI theme editor for an e-commerce FastShop platform.

You receive the current theme configuration as JSON and a user's edit instruction.
Your job is to modify ONLY the MINIMUM fields needed to fulfill the request. Do NOT change a

VISUAL MAP - What each theme_code field controls on the page:
- header_background_color: The HEADER/NAVIGATION BAR background. Use this for "make header c
- theme_background_color: The MAIN PAGE background (white area behind all content). NOT the
- button_background_color: ALL buttons, the category navigation bar background, section acce
- button_font_color: Text color INSIDE buttons and on colored accent sections.
- link_highlight_color: Link text color and hover highlights.
- thumbnail_background_color: Background behind product thumbnail images.
- product_border_color: Border around product cards (if enabled).
- product_card_bg_color: Product card background color.
- product_card_text_color: Product card text color.
- primary_font_family: Font for all text on the site.
- primary_font_size: Base font size multiplier.
- primary_font_color: Main body text color (paragraphs, descriptions).

WHAT "HEADER" MEANS: The header/navbar contains the logo, navigation links (Home, About, Cor
```

```
CONTENT FIELDS:
- content.hero_title, content.hero_subtitle, content.hero_cta_text -
Hero banner text
- content.brand_name, content.header_description, content.footer_description -
Brand info
- content.faq - Array of {question, answer}
- content.about_us - About section text

LAYOUT FIELDS:
- design_style: "modern-minimal" (light, clean) | "warm-cozy" (warm tones) | "bold-vibrant"
- layout_config.category_menu_style: "horizontal" (top bar) | "vertical" (left sidebar)
- layout_config.product_grid_columns: 2, 3, or 4 (number of products per row, default 4)
- layout_config.header_layout: "standard" (logo left, nav right) | "centered" (logo centered
- layout_config.footer_columns: 2, 3, or 4 (number of footer columns, default 4)
- layout_config.custom_css: Raw CSS string injected into the page for edge-case customizatio
- layout_config.variants.hero: Three options:
    * "split" - text on left, product image on right, side-by-side. THIS IS "horizontal" / "
    * "centered" - full-width banner with background image and text centered. THIS IS "verti
    * "card_overlay" - full-bleed image with a floating frosted card holding the text. THIS
  MAPPING RULES:
    - "horizontal" / "side by side" / "two column" / "wide" → "split"
    - "vertical" / "stacked" / "vertical cards" / "centered" / "full width" → "centered"
    - "overlay" / "floating card" / "card over image" → "card_overlay"
- layout_config.variants.categories: "carousel" (swiper slider) | "grid" (static 4-col) | "m
- layout_config.variants.products: "magazine" (default) | "wide_details" (3-col with roomier
- layout_config.variants.about: "editorial" (image left, text right) | "stacked" (image top,
- layout_config.variants.reviews: "magazine_cards" (default) | "quote_wall" (minimal with la
- layout_config.variants.footer: "classic" (4-col) | "minimal" (centered stacked)
- section_layout: Array of {type, data} controlling which sections appear

CRITICAL RULES:
1. Change the MINIMUM number of fields. If user says "make header darker", ONLY add/change t
2. If user says "change button color to red", ONLY change theme_code.button_background_color
3. NEVER change theme_background_color unless user specifically says "page background" or "s
4. NEVER change design_style unless user specifically asks to change the overall style/theme
5. NEVER change multiple fields when only one is needed.
6. You CAN add new keys to theme_code if needed (e.g., header_background_color may not exist
ADD it).
7. You CAN add the layout_config object if it doesn't exist yet.
8. Return the COMPLETE JSON with all original values preserved, only the requested field(s)
9. Return valid JSON only - no markdown, no code fences, no explanation.
```

Input fields

Field	Description
`current_theme_json`	Current complete theme configuration as JSON string
`edit_request`	User's natural language instruction for what to change

Output

Field	Description
`updated_theme_json`	Complete updated theme JSON with MINIMAL changes applied. Must be valid JSON.

Server-side variant sanitization (post-LLM)

The LLM sometimes invents invalid variant values (e.g. "horizontal" for a section that doesn't accept it). `main.py` enforces:

```
_VARIANT_ALLOWED = {
    "hero": {"split", "centered", "card_overlay"},
    "categories": {"carousel", "grid", "masonry"},
    "products": {"magazine", "wide_details", "scroll_strip"},
    "about": {"editorial", "stacked", "offset_frame"},
    "reviews": {"magazine_cards", "quote_wall", "carousel"},
    "footer": {"classic", "minimal"},
}
_VARIANT_DEFAULTS = {"hero": "split", "categories": "carousel", "products": "magazine",
                     "about": "editorial", "reviews": "magazine_cards", "footer": "classic"}
```

Invalid values fall back to the prior value if it was valid, else the default.

---

8. `/api/apply-tool` — DETERMINISTIC, NO LLM

Used by the UI's visual variant picker (chips/buttons). A click dispatches a single tool through `TOOL_HANDLERS` directly — no LLM round trip.

File: `app/main.py` (around line 1912)

Request shape:

```
{
  "tool": "<one of TOOL_HANDLERS keys>",
  "args": { ... tool-specific args ... },
  "theme_code": {...}, "content": {...}, "section_layout": [...],
```

```
  "design_style": "...", "layout_config": {...}
}
```

No prompt — the tool function is invoked directly with validated enum args. Returns the updated theme.

---

9. Quick reference — every prompt at a glance

#	Stage / Endpoint	Prompt source (file → name)	LLM call?
1	business_gathering (Node 1)	`app/workflow_nodes/business_gathering/prompts.py` → `BUSINESS_GATHERING_SYSTEM_PROMPT`	Yes (Azure `responses.create`, `text_format=`)
2	design_tokens (Node 2)	`app/workflow_nodes/design_token_node/node.py` → inline f-string in `DesignTokenGenerator.forward`	Yes (DSPy)
3	page_structure (Node 3)	`app/workflow_nodes/page_structure_node/node.py` → inline f-string in `PageStructureGenerator.forward`	Yes (DSPy)
4	content_generation (Node 4)	`app/workflow_nodes/content_generation_node/node.py` → inline f-string in `ContentGenerator.forward`	Yes (DSPy)
5	image_prompts (Node 5)	`app/workflow_nodes/image_prompt_node/node.py` → `_build_image_prompts_template`	No — template
6	`/api/edit-theme-v2`	`app/edit_agent/agent.py` → `SYSTEM_PROMPT_TEMPLATE` + `app/edit_agent/tools.py` → `TOOL_SPECS`	Yes (Azure `responses.create`)
7	`/api/edit-theme` (legacy)	`app/main.py` (inline) → `ThemeEditSignature`	Yes (DSPy)
8	`/api/apply-tool`	n/a	**No** — direct dispatch

---

10. Key design decisions that shape the prompts

These are documented in `D:\Universell Projects\universell-ai-apis\CLAUDE.md` and shape every prompt above:

- **No feature flags** — archetypes + design kit + CRM-tuning + tool-calling edit are always on. No dual-path code.
- **Reasoning-model safety** — `reasoning.effort=low` is set on every Azure call; `max_output_tokens=6000-8000` gives breathing room without burning budget on thought chains.
- **Response API only** — `chat.completions.create` returns HTTP 400 ("requested operation is unsupported") for gpt-5.1-codex-mini. All direct Azure calls use `responses.create`. DSPy nodes use DSPy's own LM wrapper.
- **User colors / content always win over LLM** — design_token_node and content_generation_node re-extract user values from the business_plan AFTER the LLM responds, and overwrite the LLM's picks.
- **Section_layout is built in Python, not by the LLM** — the LLM provides flat content; Python assembles the structured `section_layout` array. Reliably structurally correct, no schema drift.
- **`image_prompts` deliberately has no LLM call** — was 150–340s via reasoning LLM; now ~0s via template. Image GEN still happens (3 images, parallel, 90s timeout each).

Generated from the Python backend on 2026-05-13. If any prompt has been edited after this date, re-run the extraction.